

```
In [1]: import pandas as pd
df = pd.read_excel("D:\\Data Engineering\\Python\\EmployeeDataToClean.xlsx")
```

```
In [2]: df.head(2)
```

```
Out[2]:
```

	EmployeeID	NationalIDNumber	LoginID	Title	PhoneNumber	BirthDate	MaritalStatus	Gender	HireDate
0	1	14417807	adventure-works\\guy1	Gustavo Achong	9.062372e+09	1986-02-21	M	M	2008-02-28
1	2	253022876	adventure-works\\kevin0	Catherine Abel	8.086507e+09	1991-03-12	S	M	2009-03-15



```
In [3]: original_data = df # copying the data.
```

```
In [4]: df.shape
```

```
Out[4]: (50, 14)
```

```
In [5]: list(df.columns)
```

```
Out[5]: ['EmployeeID',
'NationalIDNumber',
'LoginID',
'Title',
'PhoneNumber',
'BirthDate',
'MaritalStatus',
'Gender',
'HireDate',
'Dept',
'Salary',
'Job Grade',
'CurrentFlag',
'rowguid']
```

```
In [6]: df["EmployeeID"].count()
```

```
Out[6]: 50
```

```
In [7]: df[["Dept"]]
```

Out[7]:

	Dept
0	Sales
1	Sales
2	Finance
3	Logistics
4	Human Resource
5	Sales
6	sales
7	Finance
8	Finance
9	Logistics
10	Logistics
11	Human Resource
12	Production
13	Logistics
14	Production
15	Production
16	Production
17	Sales
18	sales
19	Finance
20	Production
21	Logistics
22	Human Resource
23	Finance
24	Production
25	Production
26	Finance
27	Human Resource
28	Finance
29	Production
30	Production
31	Sales
32	Sales
33	Finance
34	Logistics
35	Human Resource
36	Production
37	Production
38	Production
39	Finance
40	sales
41	Logistics
42	Logistics

	Dept
43	Sales
44	Production
45	Production
46	Human Resource
47	Finance
48	Finance
49	sales

```
In [8]: df["Dept"].unique() # to get the unique values from the column
```

```
Out[8]: array(['Sales', 'Finance', 'Logistics', 'Human Resource', 'sales',
              'Production'], dtype=object)
```

```
In [9]: df["Gender"].unique()
```

```
Out[9]: array(['M', 'F'], dtype=object)
```

```
In [10]: df["Job Grade"].unique()
```

```
Out[10]: array(['Admin', 'Management', 'Operations'], dtype=object)
```

```
In [11]: df["Dept"].nunique() # to get distinct count of values from column
```

```
Out[11]: 6
```

```
In [12]: df['Gender'].nunique()
```

```
Out[12]: 2
```

```
In [13]: df["EmployeeID"].nunique()
```

```
Out[13]: 50
```

```
In [14]: df['NationalIDNumber'].nunique()
```

```
Out[14]: 50
```

```
In [15]: df["LoginID"].nunique()
```

```
Out[15]: 13
```

```
In [16]: df.isna().sum()
```

```
Out[16]: EmployeeID      0
NationalIDNumber    0
LoginID             37
Title               0
PhoneNumber         7
BirthDate           0
MaritalStatus       0
Gender              0
HireDate            0
Dept                0
Salary              6
Job Grade           0
CurrentFlag         0
rowguid             0
dtype: int64
```

```
In [17]: (df.isna().sum()/df.shape[0])* 100
```

```
Out[17]: EmployeeID      0.0
NationalIDNumber    0.0
LoginID             74.0
Title               0.0
PhoneNumber         14.0
BirthDate           0.0
MaritalStatus       0.0
Gender              0.0
HireDate            0.0
Dept                0.0
Salary             12.0
Job Grade           0.0
CurrentFlag         0.0
rowguid             0.0
dtype: float64
```

```
In [18]: df['CurrentFlag'].nunique()
```

```
Out[18]: 1
```

```
In [19]: df['rowguid'].nunique()
```

```
Out[19]: 50
```

```
In [20]: CurrentFlagcolumns_2_drop = ["NationalIDNumber", "CurrentFlag", "LoginID", "rowguid"]
```

```
In [21]: df = df.drop(CurrentFlagcolumns_2_drop, axis = 1) #dropping the columns
```

```
In [22]: df.shape
```

```
Out[22]: (50, 10)
```

```
In [23]: df.head(2)
```

```
Out[23]:
```

	EmployeeID	Title	PhoneNumber	BirthDate	MaritalStatus	Gender	HireDate	Dept	Salary	Job Grade
0	1	Gustavo Achong	9.062372e+09	1986-02-21	M	M	2013-02-02	Sales	2295.0	Admin
1	2	Catherine Abel	8.086507e+09	1991-03-12	S	M	2013-08-31	Sales	962.0	Management



```
In [24]: df.groupby(["Job Grade"])[["EmployeeID"]].count()
```

```
Out[24]:
```

EmployeeID	
Job Grade	
Admin	21
Management	8
Operations	21

```
In [25]: df.groupby(["Dept"])[["EmployeeID"]].count()
```

```
Out[25]:
```

EmployeeID	
Dept	
Finance	11
Human Resource	6
Logistics	8
Production	14
Sales	7
sales	4

```
In [26]: df = df.rename(columns = {"EmployeeID": "EmpID", "Title": "EmpName", 'PhoneNumber': 'PhNo'})
```

```
In [27]: df.head(2)
```

```
Out[27]:
```

	EmpID	EmpName	PhNo	BirthDate	MaritalStatus	Gender	HireDate	Dept	Salary	Job Grade
0	1	Gustavo Achong	9.062372e+09	1986-02-21	M	M	2013-02-02	Sales	2295.0	Admin
1	2	Catherine Abel	8.086507e+09	1991-03-12	S	M	2013-08-31	Sales	962.0	Management

```
In [28]: df["PhNo"] = df["PhNo"].fillna("00000000")
```

```
In [29]: df[["PhNo"]]
```

Out[29]:

	PhNo
0	9062372448.0
1	8086506867.0
2	9312378140.0
3	9467533569.0
4	00000000
5	00000000
6	00000000
7	00000000
8	8778647341.0
9	9451409795.0
10	9324393119.0
11	9234015462.0
12	9551031479.0
13	9132164070.0
14	8567094730.0
15	8653119445.0
16	8854519144.0
17	00000000
18	00000000
19	00000000
20	8780090702.0
21	9634483963.0
22	8333105168.0
23	8645474482.0
24	8567222521.0
25	9545376631.0
26	9073299916.0
27	8613707235.0
28	8463032269.0
29	8597103484.0
30	8955071753.0
31	9380032137.0
32	8617854470.0
33	9022256118.0
34	9549200442.0
35	9097595106.0
36	8964798696.0
37	8106821760.0
38	8270160632.0
39	8305392860.0
40	8176855363.0
41	9294995982.0
42	9134764648.0

	PhNo
43	8119269841.0
44	9556128856.0
45	9252359460.0
46	8679000688.0
47	8874165878.0
48	8259498695.0
49	8114919943.0

```
In [30]: df["Salary"] # will use median to fill null values
```

```
Out[30]: 0      2295.0
1       962.0
2      4006.0
3      4547.0
4      1932.0
5         NaN
6         NaN
7      3989.0
8      1636.0
9      3386.0
10     3676.0
11     1128.0
12     2301.0
13     2887.0
14     2154.0
15     2741.0
16     4096.0
17     1998.0
18     3408.0
19     3390.0
20     2034.0
21      600.0
22     4406.0
23     2042.0
24      664.0
25     3723.0
26     1161.0
27     1711.0
28     3199.0
29     1513.0
30     3978.0
31     3639.0
32     1134.0
33     2298.0
34     3843.0
35     3880.0
36     3967.0
37      561.0
38     1395.0
39      548.0
40     1355.0
41         NaN
42         NaN
43         NaN
44         NaN
45     1808.0
46     2533.0
47     3110.0
48     4491.0
49     2679.0
Name: Salary, dtype: float64
```

```
In [31]: df['Salary'] = df['Salary'].fillna(df['Salary'].median())
```

```
In [32]: df['Salary']
```

```
Out[32]: 0      2295.0
1         962.0
2      4006.0
3      4547.0
4      1932.0
5      2417.0
6      2417.0
7      3989.0
8      1636.0
9      3386.0
10     3676.0
11     1128.0
12     2301.0
13     2887.0
14     2154.0
15     2741.0
16     4096.0
17     1998.0
18     3408.0
19     3390.0
20     2034.0
21         600.0
22     4406.0
23     2042.0
24         664.0
25     3723.0
26     1161.0
27     1711.0
28     3199.0
29     1513.0
30     3978.0
31     3639.0
32     1134.0
33     2298.0
34     3843.0
35     3880.0
36     3967.0
37         561.0
38     1395.0
39         548.0
40     1355.0
41     2417.0
42     2417.0
43     2417.0
44     2417.0
45     1808.0
46     2533.0
47     3110.0
48     4491.0
49     2679.0
Name: Salary, dtype: float64
```

```
In [33]: df.isna().sum()
```

```
Out[33]: EmpID      0
EmpName    0
PhNo       0
BirthDate  0
MaritalStatus 0
Gender     0
HireDate   0
Dept       0
Salary     0
Job Grade  0
dtype: int64
```

```
In [34]: df.head()
```

```
Out[34]:
```

	EmpID	EmpName	PhNo	BirthDate	MaritalStatus	Gender	HireDate	Dept	Salary	Job Grade
0	1	Gustavo Achong	9062372448.0	1986-02-21		M	2013-02-02	Sales	2295.0	Adm
1	2	Catherine Abel	8086506867.0	1991-03-12		S	2013-08-31	Sales	962.0	Managem
2	3	Kim Abercrombie	9312378140.0	1978-09-21		M	2014-06-16	Finance	4006.0	Adm
3	4	Humberto Acevedo	9467533569.0	1978-11-01		S	2014-07-10	Logistics	4547.0	Adm
4	5	Pilar Ackerman	00000000	1963-06-07		M	2014-07-16	Human Resource	1932.0	Adm



```
In [35]: df.groupby(["Dept"])[["EmpID"]].count()
```

```
Out[35]:
```

	EmpID
Dept	
Finance	11
Human Resource	6
Logistics	8
Production	14
Sales	7
sales	4

```
In [36]: df['Dept'] = df['Dept'].str.capitalize()
```

```
In [37]: df.groupby(['Dept'])[['EmpID']].count()
```

Out[37]:

EmpID	
Dept	
Finance	11
Human resource	6
Logistics	8
Production	14
Sales	11

```
In [38]: def gender_f(a):
         if a == "M":
             return "Male"
         else:
             return "Female"
```

```
In [39]: def maritalstatus_f(a):
         if a == "S":
             return "Single"
         else:
             return "Married"
```

```
In [40]: df['Gender'] = df['Gender'].apply(gender_f)
```

```
In [41]: df['MaritalStatus'] = df['MaritalStatus'].apply(maritalstatus_f)
```

```
In [42]: df.head()
```

Out[42]:

	EmpID	EmpName	PhNo	BirthDate	MaritalStatus	Gender	HireDate	Dept	Salary	Job Grade
0	1	Gustavo Achong	9062372448.0	1986-02-21	Married	Male	2013-02-02	Sales	2295.0	Adm
1	2	Catherine Abel	8086506867.0	1991-03-12	Single	Male	2013-08-31	Sales	962.0	Manageme
2	3	Kim Abercrombie	9312378140.0	1978-09-21	Married	Male	2014-06-16	Finance	4006.0	Adm
3	4	Humberto Acevedo	9467533569.0	1978-11-01	Single	Male	2014-07-10	Logistics	4547.0	Adm
4	5	Pilar Ackerman	00000000	1963-06-07	Married	Male	2014-07-16	Human resource	1932.0	Adm

```
In [80]: df.dtypes
```

```
Out[80]: EmpID          int64
EmpName        object
PhNo           object
BirthDate      datetime64[ns]
MaritalStatus  object
Gender         object
HireDate       datetime64[ns]
Dept           object
Salary         float64
Job Grade      object
dtype: object
```

```
In [71]: df['PhNo'] = df['PhNo'].astype('int64') # to cast the data type
```

```
In [136]: df.head()
```

```
Out[136]:
```

	EmpID	EmpName	PhNo	BirthDate	MaritalStatus	Gender	HireDate	Dept	Salary	Job Grade
0	1	Gustavo Achong	9062372448	1986-02-21	Married	Male	2013-02-02	Sales	2295.0	Admin
1	2	Catherine Abel	8086506867	1991-03-12	Single	Male	2013-08-31	Sales	962.0	Management
2	3	Kim Abercrombie	9312378140	1978-09-21	Married	Male	2014-06-16	Finance	4006.0	Admin
3	4	Humberto Acevedo	9467533569	1978-11-01	Single	Male	2014-07-10	Logistics	4547.0	Admin
4	5	Pilar Ackerman	0	1963-06-07	Married	Male	2014-07-16	Human resource	1932.0	Admin



```
In [ ]: ##Phone numebr maksing 906xxxxx48 #assignment
```

```
In [73]: df['PhNo'][:3]
```

```
Out[73]: 0    9062372448
1    8086506867
2    9312378140
Name: PhNo, dtype: int64
```

```
In [77]: df['PhNo'] = df['PhNo'].astype('object')
```

```
In [68]: def mask_ph_no(phone):
return phone[:3] + 'xxxxx' + phone[-2:]
```

```
In [78]: mask_ph_no('9467533569')
```

```
Out[78]: '946xxxxx69'
```

```
In [82]: df['PhNo'] = df['PhNo'].str(phone).apply(mask_ph_no)
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[82], line 1
----> 1 df['PhNo'] = df['PhNo'].str(phone).apply(mask_ph_no)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:5989, in NDFrame.__getattr__(self, name)
    5982 if (
    5983     name not in self._internal_names_set
    5984     and name not in self._metadata
    5985     and name not in self._accessors
    5986     and self._info_axis._can_hold_identifiers_and_holds_name(name)
    5987 ):
    5988     return self[name]
-> 5989 return object.__getattribute__(self, name)

File ~\anaconda3\Lib\site-packages\pandas\core\accessor.py:224, in CachedAccessor.__get__(self, obj, cls)
    221 if obj is None:
    222     # we're accessing the attribute of the class, i.e., Dataset.geo
    223     return self._accessor
--> 224 accessor_obj = self._accessor(obj)
    225 # Replace the property with the accessor object. Inspired by:
    226 # https://www.pydanny.com/cached-property.html (https://www.pydanny.com/cached-property.html)
    227 # We need to use object.__setattr__ because we overwrite __setattr__ on
    228 # NDFrame
    229 object.__setattr__(obj, self._name, accessor_obj)

File ~\anaconda3\Lib\site-packages\pandas\core\strings\accessor.py:181, in StringMethod s.__init__(self, data)
    178 def __init__(self, data) -> None:
    179     from pandas.core.arrays.string_ import StringDtype
--> 181     self._inferred_dtype = self._validate(data)
    182     self._is_categorical = is_categorical_dtype(data.dtype)
    183     self._is_string = isinstance(data.dtype, StringDtype)

File ~\anaconda3\Lib\site-packages\pandas\core\strings\accessor.py:235, in StringMethod s._validate(data)
    232 inferred_dtype = lib.infer_dtype(values, skipna=True)
    234 if inferred_dtype not in allowed_types:
--> 235     raise AttributeError("Can only use .str accessor with string values!")
    236 return inferred_dtype
```

AttributeError: Can only use .str accessor with string values!